Proof-of-Concept Kit — Microsoft Defender for Endpoint

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Proof-of-Concept Kit: Microsoft Defender for Endpoint 5

What you get . . . 5

What this delivers . . . 5

The standard deployment model . . . 5

Boundary . . . 5

Effort . . . 6

1. Solution Overview — Proof-of-Concept Kit: Microsoft Defender for Endpoint 6

What Microsoft Defender for Endpoint is . . . 6

What a Proof-of-Concept Kit is . . . 6

Why this matters . . . 7

How it works . . . 7

What you can control . . . 7

Limitations (be honest with buyers) . . . 7

Connections and authentication used . . . 8

The standard deployment model . . . 8

2. Requirements Guide — Proof-of-Concept Kit: Microsoft Defender for Endpoint 8

2.1 Defender for Endpoint plan available to activate . . . 8

2.2 Pilot device ring (test devices only) . . . 8

2.3 Onboarding channel to the pilot devices . . . 9

2.4 A tester workstation for benign signal . . . 9

2.5 Evaluation criteria agreed up front . . . 9

2.6 Response governance for the PoC (approval-gated) . . . 9

2.7 Advanced-hunting read access . . . 9

Checklist . . . 10

Global Requirements — Defender (category-level) 10

G1. Microsoft 365 / Entra tenant . . . 10

G2. Defender licensing . . . 10

G3. Administrative roles (least privilege) . . . 10

G4. Onboarding reach to endpoints / identity / workloads . . . 11

G5. Network access to Defender service endpoints . . . 11

G6. A pilot ring . . . 11

G7. Log Analytics workspace (where required) . . . 11

G8. Response governance . . . 11

3. Implementation Guide — Proof-of-Concept Kit: Microsoft Defender for Endpoint 11

Naming (set once, reuse verbatim) . . . 12

Step 1 — Assess readiness (read-only) . . . 12
Step 2 — Enable MDE on the pilot ring . . . 12
Step 3 — Generate benign test signal . . . 13
Step 4 — Validate and demonstrate the detections . . . 13
Step 5 — Score against your criteria . . . 13
Step 6 — Tear down (clean rollback) . . . 13
Step 7 — Decide: expand or stop . . . 14
5. Testing Guide — Proof-of-Concept Kit: Microsoft Defender for Endpoint 14
Test 1 — Pilot onboarding . . . 14
Test 2 — Benign signal generated . . . 14
Test 3 — Detection reached the portal . . . 14
Test 4 — Device timeline richness . . . 14
Test 5 — Advanced-hunting visibility . . . 15
Test 6 — Automated investigation started (approval-gated) . . . 15
Test 7 — Scorecard produced . . . 15
Test 8 — Clean teardown . . . 15
6. Troubleshooting Guide — Proof-of-Concept Kit: Microsoft Defender for Endpoint 15
Pilot device not onboarding . . . 15
EICAR not detected . . . 16
No alert from test signal . . . 16
Timeline empty . . . 16
Hunting tables empty . . . 16
AIR did not start . . . 16
Automated response acted unattended . . . 16
Script cannot authenticate . . . 17
Teardown left residue . . . 17
7. Best Practices — Proof-of-Concept Kit: Microsoft Defender for Endpoint 17
Keep the PoC scoped and reversible . . . 17
Benign signal only — never real attacks . . . 17
Set the bar before you run . . . 17
Keep automated response approval-gated . . . 17
Allow warm-up before judging . . . 18
Tear down completely . . . 18
Keep licensing and preview facts current . . . 18
Defer the decisions to the owners . . . 18
Where to go next . . . 18

8. Deliverables Summary — Proof-of-Concept Kit: Microsoft Defender for Endpoint 18
Documentation 18
PowerShell scripts (Microsoft Graph Security / Intune Graph + KQL) 19
What the buyer receives 19
Outcome 19
Appendix A — PowerShell Scripts 21

Executive Summary — Proof-of-Concept Kit: Microsoft Defender for Endpoint

White-label · Defender

Deciding on Microsoft Defender for Endpoint (MDE) shouldn't rest on a slide deck. This Proof-of-Concept (PoC) Kit stands up a **scoped, safe** proof of concept in your own tenant: it enables MDE on a small **pilot ring**, generates **benign, simulated** test signal (the industry-standard EICAR test file and Microsoft's own built-in evaluation simulations — never real malware or attacks), demonstrates the headline **endpoint detection & response (EDR)** capabilities on that signal, scores the result against your evaluation criteria, and then **cleanly tears the PoC back down**.

The kit is written for an administrator with no prior Defender experience. Everything is **pilot-scoped and reversible**: you onboard a handful of test devices, prove the value, capture a scorecard, and offboard — with nothing left behind on the wider estate.

What you get

- A plain-language **Solution Overview** of MDE, EDR, and how a PoC proves it on real-but-safe signal.
- A **Requirements Guide** of the PoC-specific prerequisites (the category-wide ones live in `global-requirements-defender.md`).
- An **Implementation Guide** — exact, ordered steps to enable MDE on the pilot ring, generate benign signal, demonstrate the detections, score, and tear down.
- Five **PowerShell scripts** (Microsoft Graph Security + Intune Graph) with **KQL advanced-hunting** queries: a `-WhatIf`-gated pilot enable, a **benign** test-signal generator, a demo/validate check, a criteria scorecard, and a `-WhatIf`-gated teardown.
- **Testing**, **Troubleshooting**, and **Best Practices** guides.

What this delivers

A concrete, **in-tenant proof** that Microsoft Defender for Endpoint delivers EDR — an onboarded pilot device raising a real detection from benign test signal, the alert visible in the incident queue, device timeline, and advanced hunting, an **automated investigation** kicked off (kept approval-gated), and a **scorecard** mapping each observed capability to your pass/fail criteria — all torn down cleanly at the end.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — here scoped as a PoC: assess readiness, enable MDE on a pilot ring, generate benign signal, validate and score, then either expand or tear down.

Boundary

This is an **evaluation kit**, not a production rollout. It proves MDE on a pilot ring and then removes it. Production onboarding at scale, SIEM integration, and long-term hunting are separate adjacent solutions (see *Where to go next*). All test signal is **benign and simulated**. Response actions are kept **approval-gated**; incident-response and automated-response guardrail decisions remain the SOC/security

team's.

Effort

A competent administrator can complete the full PoC — enable, signal, validate, score, and teardown — in a short, focused engagement on a few pilot devices. No production change is required, and every step is reversible.

1. Solution Overview — Proof-of-Concept Kit: Microsoft Defender for Endpoint

What Microsoft Defender for Endpoint is

Microsoft Defender for Endpoint (MDE) is Microsoft's **endpoint detection & response (EDR)** platform. Onboarded devices (Windows, macOS, Linux, iOS, Android) stream behavioural telemetry to the cloud service and are visible in the unified Defender portal at `security.microsoft.com`. MDE combines:

- **Next-generation antivirus (NGAV)** — Microsoft Defender Antivirus real-time protection and cloud-delivered protection.
- **Endpoint detection & response (EDR)** — behavioural detections that raise alerts and correlate them into incidents, with a full **device timeline**.
- **Automated investigation & response (AIR)** — automated triage that can investigate and (when permitted) remediate.
- **Threat & vulnerability management (Microsoft Defender Vulnerability Management / MDVM)** — device weakness and exposure signal. *Some MDVM features are plan-gated — verify current availability in your tenant.*
- **Attack surface reduction (ASR)** and **advanced hunting** over the raw telemetry with KQL.

What a Proof-of-Concept Kit is

A PoC Kit is a **customer-runnable evaluation**, not a production deployment and not a sales demo. Its job is to answer one question — *does MDE actually detect and surface a threat on our devices?* — with evidence, safely, in your own tenant. It does that by:

1. **Enabling** MDE on a small **pilot ring** of test devices only.
2. **Generating benign test signal** — the industry-standard **EICAR** anti-malware test file and Microsoft's own **built-in evaluation simulations** (Defender portal → *Endpoints → Evaluation & tutorials*). These are documented, harmless triggers designed exactly for this purpose. **No real malware, exploit, or attack is ever used.**
3. **Demonstrating** that the signal produces a real detection — an alert in the incident queue, an entry on the device timeline, a row in advanced hunting, and an automated investigation.
4. **Scoring** what you observed against your evaluation criteria.
5. **Tearing down** — offboarding the pilot devices and removing everything the PoC created.

Why this matters

Buying or expanding an EDR is a significant decision, and a vendor slide deck can't tell you how the product behaves on *your* devices, with *your* configuration and *your* analysts. A scoped PoC replaces "trust the deck" with "watch it fire." Because it is **pilot-scoped and fully reversible**, you get that proof without risk to the production estate and without committing to a rollout before you've seen the value.

How it works

1. **Assess** readiness with `POCMDE-Enable.ps1 -WhatIf` (read-only preview) — licensed MDE plan, admin roles, the onboarding channel, and a defined pilot ring.

2. **Configure & enable** MDE on the pilot device group `MDE PoC – Pilot Ring – Pilot` via an Intune EDR onboarding policy — scoped to pilot devices only.

3. **Generate benign signal** with `POCMDE-TestSignal.ps1` — guidance and safe helpers to drop the EICAR test file and launch a Microsoft built-in simulation on a pilot device.

4. **Validate & demonstrate** with `POCMDE-Validate.ps1` — confirm the detection reached the portal, timeline, and hunting tables, and that AIR started (kept approval-gated).

5. **Score** with `POCMDE-Scorecard.ps1` against your criteria (detection fired, latency, timeline richness, hunting visibility, AIR behaviour).

6. **Tear down** with `POCMDE-Teardown.ps1 -WhatIf` first, then for real — offboard pilot devices, remove the Intune policy, delete test files, resolve test alerts.

What you can control

• **Pilot ring** — exactly which test devices onboard; nothing else is touched.

• **Which capabilities to prove** — EDR alerting, device timeline, advanced hunting, AIR, MDVM, ASR.

• **Response posture** — AIR kept in an **approval/simulate** posture throughout the PoC.

• **Scorecard criteria** — the pass/fail bar you evaluate against.

• **Teardown** — full, reversible removal.

Limitations (be honest with buyers)

• MDE and several features are **license-gated** (Defender for Endpoint **P1/P2**, MDVM add-on). *Verify current plan availability in your tenant.*

• Test signal is **benign and simulated only** — EICAR and Microsoft's built-in simulations. This proves the detection pipeline, not your resilience to a real adversary (that is a red-team engagement, out of scope).

• Advanced-hunting tables and Secure Score **populate over time** — allow warm-up after onboarding before judging.

• This is a **PoC**, not a production rollout — it deliberately stays small and is designed to be removed. Scaling out is a separate onboarding solution.

• EDR improves detection and response; it does not replace patching, backup, or the SOC's incident-response decisions.

Connections and authentication used

• The **unified Defender portal** (`security.microsoft.com`) for activation, incidents, timeline, evaluation simulations, and Secure Score.

• **Microsoft Graph Security** and the **Microsoft 365 Defender** advanced-hunting API (via `Microsoft.Graph` PowerShell modules) for read-only discovery, validation, and scoring.

• **Intune / Microsoft Endpoint Manager** Graph as the pilot onboarding channel (EDR policy assignment).

• Least-privilege roles: **Security Administrator** for configuration, **Security Reader** for read-only work, **Intune Administrator** (scoped) for the pilot policy, and **Global Administrator** only for one-time first-service activation.

The standard deployment model

> ***Assess readiness → configure/enable MDE on a pilot ring → generate benign test signal → validate the detections and score → expand or tear down.*** *Repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand****, run here as a safe, reversible PoC with automated response kept approval-gated.*

2. Requirements Guide — Proof-of-Concept Kit: Microsoft Defender for Endpoint

Verify **every** requirement before enabling anything. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant, licensing, admin roles, onboarding reach, network access, pilot ring, workspace, response governance) are in `global-requirements-defender.md` — do not repeat them; the items below are **specific to running a scoped MDE proof of concept**. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

2.1 Defender for Endpoint plan available to activate

Description: A licensed **Defender for Endpoint P1 or P2** plan the tenant can activate for the PoC. **Why:** EDR, AIR, and advanced hunting are the capabilities being proven; without the plan MDE cannot be initialized or will silently limit features. **Verify:** Admin center → **Billing** → **Licenses** shows a Defender for Endpoint plan; `security.microsoft.com` → **Settings** → **Endpoints** loads. Run `POCMDE-Enable.ps1 -WhatIf`. *Verify current plan features (P1 vs P2, MDVM add-on) in your tenant.* **Implement-if-missing:** Assign the Defender for Endpoint plan (often bundled in M365 E5 / E5 Security) before enabling.

2.2 Pilot device ring (test devices only)

Description: A small set of **non-production test devices** to onboard, grouped as `MDE PoC – Pilot Ring – Pilot`. **Why:** The PoC must touch only disposable test devices so it can be torn down with zero production impact; benign signal is generated on these devices. **Verify:** An Entra/Intune device group `MDE PoC – Pilot Ring – Pilot` exists with test devices; `POCMDE-Enable.ps1 -WhatIf` lists membership. **Implement-if-missing:** Create the group and add one to three test devices (a spare laptop or a test VM is ideal).

2.3 Onboarding channel to the pilot devices

Description: A management channel that reaches the pilot devices — **Intune** (default), Configuration Manager, Group Policy, or the local onboarding script. **Why:** MDE only collects EDR signal from onboarded devices; the channel must reach the pilot ring to enable, and to offboard at teardown. **Verify:** Pilot devices are Intune-enrolled (or reachable by GPO/script); `POCMDE-Enable.ps1 -WhatIf` reports onboarding readiness. **Implement-if-missing:** Enrol the test devices in Intune, or prepare the local onboarding/offboarding scripts from **Settings** → **Endpoints** → **Onboarding**.

2.4 A tester workstation for benign signal

Description: Interactive access to a pilot device where a tester can drop the **EICAR** test file and run a Microsoft built-in simulation. **Why:** The benign signal must be generated **on** an onboarded device to produce the detection; this is done by hand or with the safe helper in `POCMDE-TestSignal.ps1`. **Verify:** You can sign in interactively to a device in `MDE PoC – Pilot Ring – Pilot`. **Implement-if-missing:** Ensure at least one pilot device is accessible (console, RDP, or physical) for the tester.

2.5 Evaluation criteria agreed up front

Description: A short, written list of what "pass" means — e.g. *detection fires within N minutes, appears on the device timeline, is queryable in advanced hunting, and triggers an automated investigation.* **Why:** A PoC is only conclusive if the bar is set **before** you run it; the scorecard maps observations to these criteria. **Verify:** A criteria list exists; `POCMDE-Scorecard.ps1` accepts it as input. **Implement-if-missing:** Draft 5–8 criteria with the evaluating stakeholders before Step 1. *What counts as "good enough" is the customer's decision.*

2.6 Response governance for the PoC (approval-gated)

Description: Agreement that **automated investigation & response (AIR)** stays in an **approval/semi** posture for the duration of the PoC. **Why:** Even in a pilot, AIR can take actions (isolate, quarantine); the PoC proves detection without letting automation act unattended (per `global-requirements-defender.md` G8). **Verify:** AIR automation level for the pilot device group is **Semi — require approval**; `POCMDE-Validate.ps1` reports it. **Implement-if-missing:** Set the pilot group's automation level to **Semi — require approval** before generating signal. *The automation-guardrail decision is the SOC/security team's.*

2.7 Advanced-hunting read access

Description: Permission to run advanced-hunting queries via the portal and the Graph Security API (`ThreatHunting.Read.All`). **Why:** The validate and scorecard scripts confirm the detection landed in the hunting tables (`DeviceEvents`, `AlertInfo`, `AlertEvidence`). **Verify:**

`security.microsoft.com` → **Hunting** → **Advanced hunting** runs a query; `POCMDE-Validate.ps1` authenticates and returns rows. **Implement-if-missing:** Grant the running account **Security Reader** (or higher) and consent to the read-only hunting scope.

Checklist

- Defender for Endpoint plan available to activate — `POCMDE-Enable.ps1 -WhatIf`
- Pilot device ring `MDE PoC – Pilot Ring – Pilot` (test devices only)
- Onboarding channel reaches the pilot devices (Intune/GPO/script)
- A tester workstation for benign EICAR / simulation signal
- Evaluation criteria agreed and written down — `POCMDE-Scorecard.ps1`
- AIR kept approval-gated for the PoC — `POCMDE-Validate.ps1`
- Advanced-hunting read access (`ThreatHunting.Read.All`)

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/ users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles`

`and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Proof-of-Concept Kit: Microsoft Defender for Endpoint

Follow these steps **in order**. Most work is in the **unified Defender portal** (`https://security.microsoft.com`); pilot onboarding uses the **Intune admin center** (`https://intune.microsoft.com`) as the default channel. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model (run as a PoC):*** *Assess readiness, enable MDE on a **pilot ring** of test devices only, generate **benign** test signal, validate the detections and score them, then **tear down**. All test signal is EICAR or a Microsoft built-in simulation — **never real malware or attacks**. Keep **AIR approval-gated** throughout. Nothing production is changed.*

Naming (set once, reuse verbatim)

- Pilot device group (Entra/Intune): `MDE PoC – Pilot Ring – Pilot`
- Intune EDR onboarding policy: `MDE PoC EDR Onboarding – Pilot Ring – Pilot`
- Scorecard output file: `MDE PoC Scorecard – Pilot Ring – Validate`
- Test-signal run tag (used in notes/hunting): `MDE-PoC-TESTSIGNAL`

Step 1 — Assess readiness (read-only)

1. Sign in to `security.microsoft.com` as a **Security Administrator**.
2. Run `POCMDE-Enable.ps1 -WhatIf` — it previews (changes nothing): the licensed MDE plan, admin roles, whether MDE is already activated, the onboarding channel, and whether `MDE PoC – Pilot Ring – Pilot` exists.
3. In **Entra** (`entra.microsoft.com`) → **Groups**, create the device group `MDE PoC – Pilot Ring – Pilot` and add one to three **test** devices only.

Step 2 — Enable MDE on the pilot ring

1. `security.microsoft.com` → **Settings** → **Endpoints** → **Onboarding**. If MDE is not yet initialized, complete the one-time activation (Global Administrator, first time only).
2. `intune.microsoft.com` → **Endpoint security** → **Microsoft Defender for Endpoint** — confirm **Connection status: Enabled**.
3. **Endpoint security** → **Endpoint detection and response** → **Create Policy** (platform: Windows). Name it `MDE PoC EDR Onboarding – Pilot Ring – Pilot`. Set **client configuration package type = Auto from connector**. **Assignments:** assign **only** to `MDE PoC – Pilot Ring – Pilot`. **Review + create.**
4. Alternatively, run `POCMDE-Enable.ps1` (without `-WhatIf`) to assign the EDR policy to the pilot ring — it is `-WhatIf`-gated and shows exactly what it will do before you confirm.
5. On a pilot device, confirm onboarding: `security.microsoft.com` → **Assets** → **Devices** shows the device as **Onboarded**.
6. `security.microsoft.com` → **Settings** → **Endpoints** → **Advanced features** — confirm **Automated Investigation** is on and set the pilot group's **automation level** to **Semi — require approval**.

Step 3 — Generate benign test signal

1. Run `POCMDE-TestSignal.ps1` from an onboarded pilot device (or follow its printed manual steps). It performs only **benign, documented** actions and tags them `MDE-PoC-TESTSIGNAL`:

• Drops the **EICAR** anti-malware test file (the standard 68-byte harmless string) to trigger a Microsoft Defender Antivirus detection.

• Prints the exact portal path to launch a **Microsoft built-in EDR simulation**: `security.microsoft.com` → **Endpoints** → **Evaluation & tutorials** → **Tutorials & simulations** → run a **"Do-it-yourself" / tutorial** scenario against the pilot device. *These simulations are provided by Microsoft specifically for safe evaluation — verify the current catalogue in your tenant.*

1. **Do not** download real malware, run exploit tools, or disable any control to "test." EICAR and the built-in simulations are sufficient and safe.

2. Note the time you generated each signal — you will compare against detection time in the scorecard.

Step 4 — Validate and demonstrate the detections

1. Run `POCMDE-Validate.ps1` — it confirms (read-only) that the benign signal produced: an **alert** (`security.microsoft.com` → **Incidents & alerts**), a **device timeline** entry, rows in advanced hunting (`DeviceEvents`, `AlertInfo`, `AlertEvidence`), and that an **automated investigation** started (kept approval-gated).

2. In the portal, walk the demonstration: open the incident → view the **alert story** → open the **device page** → **Timeline** → open **Automated investigations** → run an **Advanced hunting** query on the pilot device. This is the headline-capability demo for stakeholders.

3. Confirm AIR did **not** auto-act (it is Semi — require approval).

Step 5 — Score against your criteria

1. Run `POCMDE-Scorecard.ps1 -CriteriaPath .\criteria.json -OutFile "MDE PoC Scorecard – Pilot Ring – Validate.json"`. It maps each observation (detection fired, latency, timeline present, hunting visible, AIR started) to your agreed pass/fail criteria and writes the scorecard.

2. Review the scorecard with stakeholders. This is the decision artefact — proof on real-but-safe signal, scored against the bar you set in requirements.

Step 6 — Tear down (clean rollback)

1. Run `POCMDE-Teardown.ps1 -WhatIf` first to preview every removal, then run it for real to confirm.

2. Teardown: **offboard** the pilot devices (`security.microsoft.com` → **Settings** → **Endpoints** → **Offboarding**, or the Intune offboarding policy), **remove** the Intune policy `MDE PoC EDR Onboarding – Pilot Ring – Pilot`, **delete** the EICAR test file from the pilot devices, and **resolve** the test alerts (annotate them `MDE-PoC-TESTSIGNAL`).

3. Optionally delete the pilot group `MDE PoC – Pilot Ring – Pilot`. Keep the scorecard file as your record.

Step 7 — Decide: expand or stop

1. If the PoC passed and you choose to proceed, move to a **production onboarding** — that is a separate, adjacent solution (see *Where to go next* in Best Practices); do **not** scale from the PoC ring.
2. If not proceeding, confirm teardown is complete with `POCMDE-Validate.ps1` (it should now report no onboarded pilot devices).

Order of operations

`Assess (-WhatIf) → enable MDE on pilot ring → generate benign EICAR/simulation signal → validate + demonstrate → score vs criteria → teardown (-WhatIf then real) → decide expand or stop`

5. Testing Guide — Proof-of-Concept Kit: Microsoft Defender for Endpoint

Run these checks during the PoC (Steps 2–6 of the Implementation Guide). Each test has a **pass/fail gate**. All signal is **benign** (EICAR / Microsoft built-in simulation). Objects are referenced by the exact names from the Implementation Guide.

Test 1 — Pilot onboarding

Procedure: Run `POCMDE-Enable.ps1 -WhatIf`, then after enabling, in `security.microsoft.com` → **Assets** → **Devices**, confirm each device in `MDE PoC – Pilot Ring – Pilot` shows **Onboarded**. **Pass:** Every pilot device is onboarded and reporting. **Fail:** A device stays "Can be onboarded" — see Troubleshooting *Pilot device not onboarding*.

Test 2 — Benign signal generated

Procedure: Run `POCMDE-TestSignal.ps1` on a pilot device; confirm the EICAR file was written and quarantined, and that a built-in simulation was launched. **Pass:** Microsoft Defender Antivirus detects/quarantines EICAR; the simulation runs. Only benign signal was used. **Fail:** No AV reaction — see Troubleshooting *EICAR not detected*.

Test 3 — Detection reached the portal

Procedure: Run `POCMDE-Validate.ps1`. In **Incidents & alerts**, confirm an alert from the pilot device (tagged/annotated `MDE-PoC-TESTSIGNAL`). **Pass:** A real alert appears for the benign signal within your agreed latency window. **Fail:** No alert — see Troubleshooting *No alert from test signal*.

Test 4 — Device timeline richness

Procedure: Open the pilot **device page** → **Timeline** and confirm the EICAR/simulation activity is visible with process and file detail. **Pass:** The timeline shows the test activity with context (process tree, file, user). **Fail:** Timeline empty or sparse — see Troubleshooting *Timeline empty*.

Test 5 — Advanced-hunting visibility

Procedure: `POCMDE-Validate.ps1` runs KQL over `DeviceEvents`, `AlertInfo`, and `AlertEvidence` for the pilot device. **Pass:** Recent rows return for the pilot device covering the test signal. **Fail:** Tables empty — allow warm-up time; see Troubleshooting *Hunting tables empty*.

Test 6 — Automated investigation started (approval-gated)

Procedure: Open **Incidents** → **the test incident** → **Automated investigations**; `POCMDE-Validate.ps1` reports AIR status and automation level. **Pass:** An automated investigation started and the pilot group's level is **Semi — require approval** (no unattended action taken). **Fail:** No AIR, or automation acted unattended — see Troubleshooting *AIR did not start* / reset per Implementation Step 2.

Test 7 — Scorecard produced

Procedure: Run `POCMDE-Scorecard.ps1` and confirm `MDE PoC Scorecard – Pilot Ring – Validate.json` is written with a pass/fail per criterion. **Pass:** A complete scorecard maps every observation to your criteria. **Fail:** Missing observations — re-run the relevant validate step first.

Test 8 — Clean teardown

Procedure: Run `POCMDE-Teardown.ps1 -WhatIf`, then for real; re-run `POCMDE-Validate.ps1`. **Pass:** Pilot devices are offboarded, the Intune policy removed, EICAR deleted, test alerts resolved — validate reports nothing left onboarded. **Fail:** Residue remains — see Troubleshooting *Teardown left residue*.

Exit gate

Tests 1–7 pass to call the PoC conclusive; Test 8 passes to close it out cleanly. Keep the scorecard file as the decision record.

6. Troubleshooting Guide — Proof-of-Concept Kit: Microsoft Defender for Endpoint

Common field issues, in **symptom** → **cause** → **fix** form. Run `POCMDE-Enable.ps1 -WhatIf` and `POCMDE-Validate.ps1` first — most causes surface there. All test signal is benign (EICAR / built-in simulation).

Pilot device not onboarding

Symptom: A device in `MDE PoC – Pilot Ring – Pilot` stays "Can be onboarded" in **Assets** → **Devices**. **Cause:** The Intune EDR policy `MDE PoC EDR Onboarding – Pilot Ring – Pilot` is not

assigned, the MDE-Intune connector is off, or the device cannot reach the Defender service endpoints. **Fix:** Confirm the device is in the pilot group; confirm **Endpoint security** → **Microsoft Defender for Endpoint** → **Connection status: Enabled**; verify outbound HTTPS to the Defender endpoint URLs (per `global-requirements-defender.md` G5). Force an Intune sync and re-run `POCMDE-Enable.ps1 -WhatIf`.

EICAR not detected

Symptom: The EICAR test file is written but Microsoft Defender Antivirus does not detect/quarantine it. **Cause:** Real-time protection is off, third-party AV is primary, tamper/exclusions suppress it, or the file was altered. **Fix:** Confirm Microsoft Defender Antivirus real-time and cloud-delivered protection are on for the pilot device; remove any test-path exclusion; ensure the EICAR string is exact and unmodified. Re-run `POCMDE-TestSignal.ps1`.

No alert from test signal

Symptom: Test 3 fails — no alert appears in **Incidents & alerts** for the pilot device. **Cause:** The device is not fully onboarded, signal has not propagated yet, or the simulation targeted the wrong device. **Fix:** Confirm Test 1 passed; allow propagation time (minutes); confirm the simulation/EICAR ran on an onboarded pilot device. Re-run `POCMDE-Validate.ps1`.

Timeline empty

Symptom: The device **Timeline** shows no test activity. **Cause:** Onboarding is too recent, or the device stopped reporting. **Fix:** Allow warm-up time; confirm the device is **Onboarded** and healthy in **Assets** → **Devices**; generate the benign signal again and re-check.

Hunting tables empty

Symptom: `POCMDE-Validate.ps1` returns no rows from `DeviceEvents` / `AlertInfo` / `AlertEvidence`. **Cause:** Advanced-hunting tables populate over time, or the account lacks hunting read scope. **Fix:** Allow warm-up (tables populate over minutes to hours); confirm `ThreatHunting.Read.All` consent and **Security Reader**; re-run the validate script.

AIR did not start

Symptom: Test 6 fails — no automated investigation for the test incident. **Cause:** Automated Investigation is off, the device is not in an automation-enabled group, or the alert type did not trigger AIR. **Fix:** Confirm **Settings** → **Endpoints** → **Advanced features** → **Automated Investigation** is on and the pilot group's level is **Semi — require approval**; some benign simulations trigger AIR, some do not — use a built-in EDR simulation known to raise an EDR alert. Re-run `POCMDE-Validate.ps1`.

Automated response acted unattended

Symptom: A pilot device was isolated or a file quarantined automatically during the PoC. **Cause:** The pilot group's AIR **automation level** is higher than **Semi — require approval**. **Fix:** Reset to **Semi — require approval** (Implementation Step 2), confirm with the SOC. *The automation-guardrail decision is the security team's.*

Script cannot authenticate

Symptom: `POCMDE-*.ps1` fails on connect or returns access-denied. **Cause:** The Microsoft Graph modules are missing, read scopes were not consented, or the account lacks the role. **Fix:** Let the script install the modules (it checks on run); sign in with `Connect-MgGraph` and consent to the read-only scopes; confirm **Security Reader** (or Security Administrator for enable/teardown). Re-run.

Teardown left residue

Symptom: After `POCMDE-Teardown.ps1`, a device still shows onboarded or the Intune policy remains. **Cause:** Offboarding is asynchronous, or the teardown ran in `-WhatIf` only. **Fix:** Run teardown **without** `-WhatIf` to apply; offboarding can take time to reflect in **Assets** → **Devices** — allow it, then confirm with `POCMDE-Validate.ps1`. Remove the Intune policy and pilot group manually if any remain.

7. Best Practices — Proof-of-Concept Kit: Microsoft Defender for Endpoint

Field-proven guidance for running a clean, safe, conclusive MDE proof of concept.

Keep the PoC scoped and reversible

- Onboard **test devices only** into `MDE PoC – Pilot Ring – Pilot`. Never point a PoC at production endpoints — the whole value is that you can tear it down with zero blast radius.
- Preview every change with `-WhatIf` first (`POCMDE-Enable.ps1`, `POCMDE-Teardown.ps1`). Confirm the plan before you apply it.

Benign signal only — never real attacks

- Use **EICAR** and Microsoft's **built-in evaluation simulations** exclusively. They are designed for safe evaluation and are enough to prove the detection pipeline.
- **Do not** download live malware, run exploit frameworks, or disable a control to "see what happens." A PoC proves the product works; it is not a red-team exercise. Adversary emulation belongs in a separate, governed engagement.

Set the bar before you run

- Agree the **evaluation criteria** in writing first (Requirement 2.5). A PoC that defines "pass" afterwards proves nothing. Feed those criteria into `POCMDE-Scorecard.ps1` so the result is objective.

Keep automated response approval-gated

- Leave **AIR** at **Semi — require approval** for the entire PoC. You want to *see* the automated investigation start, not have it isolate a device unattended. Moving toward higher automation is a production decision for the SOC/security team.

Allow warm-up before judging

• Advanced-hunting tables (`DeviceEvents`, `AlertInfo`, `AlertEvidence`) and Secure Score populate over **minutes to hours**. Give the pipeline time before scoring a "fail" — use `POCMDE-Validate.ps1` to check when signal has arrived.

Tear down completely

• Run teardown and then **confirm** with `POCMDE-Validate.ps1` that nothing is left onboarded. Delete the EICAR file, resolve the `MDE-PoC-TESTSIGNAL` alerts, remove the Intune policy, and (optionally) the pilot group. Keep only the scorecard as your record.

Keep licensing and preview facts current

• Defender for Endpoint **P1/P2** and **MDVM** features are license-gated, and Microsoft's simulation catalogue changes. **Verify current availability and plan requirements in your tenant** before assuming a capability is included.

Defer the decisions to the owners

• Whether the PoC "passed," how to weight the criteria, and whether to proceed to production are **the customer's** decisions. Automated-response guardrails and incident-response runbooks are the **SOC/security team's**. This kit delivers the technical proof, not the verdict.

Where to go next

Once the PoC proves MDE, natural next steps in the catalogue:

• The **Defender XDR Deployment & Unified Onboarding** solution to move from pilot to a real, correlated production rollout across all four Defender pillars.

• The **Defender for Endpoint onboarding accelerator / Intune endpoint-security** solutions to scale onboarding properly (not from the PoC ring).

• The **Defender for Office 365 (P2) — Threat Explorer, Attack Simulation & AIR** and **Threat Hunting** solutions to deepen detection and hunting.

• The **Sentinel SOC/XDR integration** solution to stream MDE into a SIEM for long-term hunting and SOAR.

8. Deliverables Summary — Proof-of-Concept Kit: Microsoft Defender for Endpoint

Documentation

Document	Purpose
Executive Summary	What the PoC proves, the boundary, and the effort
1. Solution Overview	What MDE / EDR is and how a scoped, safe PoC proves it

Document	Purpose
2. Requirements Guide	PoC-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps — enable on a pilot ring, generate benign signal, validate, score, tear down
5. Testing Guide	Eight validation tests with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Safe-PoC guidance and where to go next

PowerShell scripts (Microsoft Graph Security / Intune Graph + KQL)

Script	What it does
`POCMDE-Enable.ps1`	`-WhatIf`-gated pilot enable — previews readiness, then assigns the Intune EDR onboarding policy to `MDE PoC – Pilot Ring – Pilot` (safe preview by default)
`POCMDE-TestSignal.ps1`	**Benign** test-signal generator — drops the EICAR test file and prints the exact steps to launch a Microsoft built-in simulation (no real malware, ever)
`POCMDE-Validate.ps1`	Read-only demo/validate — confirms the alert, device timeline, advanced-hunting rows, and that AIR started (approval-gated)
`POCMDE-Scorecard.ps1`	Scores observations against your evaluation criteria and writes `MDE PoC Scorecard – Pilot Ring – Validate`
`POCMDE-Teardown.ps1`	`-WhatIf`-gated clean rollback — offboards pilot devices, removes the Intune policy, deletes EICAR, resolves test alerts

What the buyer receives

- This solution as a **Word** document and **PDF**.
- All five **PowerShell scripts** (with embedded KQL) in a single ZIP.
- A repeatable process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — run here as a safe, reversible PoC with automated response kept approval-gated.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A concrete, in-tenant proof that Microsoft Defender for Endpoint delivers EDR — a pilot device raising a real detection from benign test signal, visible in the incident queue, device timeline, and advanced hunting, with an approval-gated automated investigation — scored against your criteria and cleanly torn down.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

POCMDE-Enable.ps1

Purpose: Stands up the MDE proof of concept on a small pilot ring ONLY. It previews readiness (licensed plan, admin roles, MDE activation state, onboarding channel, pilot group membership) and then assigns the Intune EDR onboarding policy to the pilot device group. Every change is gated behind -WhatIf: run with -WhatIf first to preview, then without -WhatIf to apply. Touches test devices only; nothing production is changed. HOW TO RUN Preview only (changes nothing): powershell -ExecutionPolicy Bypass -File .\POCMDE-Enable.ps1 -WhatIf Apply to the pilot ring: powershell -ExecutionPolicy Bypass -File .\POCMDE-Enable.ps1 -PilotGroup "MDE PoC - Pilot Ring - Pilot" Requires Microsoft.Graph.Authentication and consent to read directory + manage the pilot Intune policy (DeviceManagementConfiguration.ReadWrite.All). EXPECTED RESULTS A readiness summary, then (when applied) the Intune EDR policy assigned to the pilot group so the test devices begin onboarding to MDE. .NOTES White-label. Read-only by default (-WhatIf). Pilot-scoped. Reverse with POCMDE-Teardown.ps1. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$PilotGroup = 'MDE PoC - Pilot Ring - Pilot', [string]$EdrPolicyName = 'MDE PoC EDR Onboarding - Pilot Ring - Pilot')

How to run: Preview only (changes nothing): powershell -ExecutionPolicy Bypass -File .\POCMDE-Enable.ps1 -WhatIf Apply to the pilot ring: powershell -ExecutionPolicy Bypass -File .\POCMDE-Enable.ps1 -PilotGroup "MDE PoC - Pilot Ring - Pilot" Requires Microsoft.Graph.Authentication and consent to read directory + manage the pilot Intune policy (DeviceManagementConfiguration.ReadWrite.All). EXPECTED RESULTS A readiness summary, then (when applied) the Intune EDR policy assigned to the pilot group so the test devices begin onboarding to MDE. .NOTES White-label. Read-only by default (-WhatIf). Pilot-scoped. Reverse with POCMDE-Teardown.ps1. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$PilotGroup = 'MDE PoC - Pilot Ring - Pilot', [string]$EdrPolicyName = 'MDE PoC EDR Onboarding - Pilot Ring - Pilot')

POCMDE-Scorecard.ps1

Purpose: Turns the PoC observations into an objective, shareable scorecard. It re-checks the key MDE capabilities (detection reached the portal, evidence attached, device timeline/hunting populated, automated investigation started) via read-only advanced hunting, maps each to the pass/fail criteria you agreed up front, and writes a JSON + console scorecard. Provide your criteria file (see the sample below) so the bar is yours, not ours. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\POCMDE-Scorecard.ps1 ` -CriteriaPath .\criteria.json -OutFile "MDE PoC Scorecard - Pilot Ring - Validate.json" Sample criteria.json: [{ "id":"detect", "label":"Detection reached portal", "capability":"alert", "required":true }, { "id":"evidence", "label":"Evidence attached to alert", "capability":"evidence", "required":true }, { "id":"timeline", "label":"Device timeline populated", "capability":"device", "required":true }, { "id":"air", "label":"Automated investigation start", "capability":"air", "required":false }] Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A per-criterion PASS/FAIL table, an overall pass rate, and a saved scorecard file. .NOTES White-label. Read-only. Whether the PoC "passed" and how criteria are weighted is the customer's decision. > [CmdletBinding()] param([string]$CriteriaPath, [string]$OutFile = 'MDE PoC Scorecard - Pilot Ring -

Validate.json', [string]$DeviceName, [int]$LookbackHours = 12)

How to run: powershell -ExecutionPolicy Bypass -File .\POCMDE-Scorecard.ps1 ` -CriteriaPath .\criteria.json -OutFile "MDE PoC Scorecard - Pilot Ring - Validate.json" Sample criteria.json: [{ "id":"detect", "label":"Detection reached portal", "capability":"alert", "required":true }, { "id":"evidence", "label":"Evidence attached to alert", "capability":"evidence", "required":true }, { "id":"timeline", "label":"Device timeline populated", "capability":"device", "required":true }, { "id":"air", "label":"Automated investigation start", "capability":"air", "required":false }] Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A per-criterion PASS/FAIL table, an overall pass rate, and a saved scorecard file. .NOTES White-label. Read-only. Whether the PoC "passed" and how criteria are weighted is the customer's decision. > [CmdletBinding()] param([string]$CriteriaPath, [string]$OutFile = 'MDE PoC Scorecard - Pilot Ring - Validate.json', [string]$DeviceName, [int]$LookbackHours = 12)

POCMDE-Teardown.ps1

Purpose: Removes everything the PoC created so the tenant returns to its pre-PoC state. It: - removes the Intune EDR onboarding policy assignment from the pilot ring (and optionally deletes the policy), - prints the exact offboarding path for the pilot devices, - deletes the benign EICAR test file from the local machine, - reminds you to resolve the MDE-PoC-TESTSIGNAL test alerts. Every change is gated behind -WhatIf: run with -WhatIf first to preview, then without to apply. HOW TO RUN Preview: powershell -ExecutionPolicy Bypass -File .\POCMDE-Teardown.ps1 -WhatIf Apply: powershell -ExecutionPolicy Bypass -File .\POCMDE-Teardown.ps1 -DeletePolicy Requires Microsoft.Graph.Authentication and DeviceManagementConfiguration.ReadWrite.All. EXPECTED RESULTS The pilot EDR policy assignment removed, the EICAR file deleted, and printed steps to offboard the pilot devices and resolve the test alerts. .NOTES White-label. Reversible teardown. Offboarding devices is asynchronous — allow time to reflect. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$PilotGroup = 'MDE PoC - Pilot Ring - Pilot', [string]$EdrPolicyName = 'MDE PoC EDR Onboarding - Pilot Ring - Pilot', [switch]$DeletePolicy, [string]$WorkDir = "$env:TEMP\MDE-PoC")

How to run: Preview: powershell -ExecutionPolicy Bypass -File .\POCMDE-Teardown.ps1 -WhatIf Apply: powershell -ExecutionPolicy Bypass -File .\POCMDE-Teardown.ps1 -DeletePolicy Requires Microsoft.Graph.Authentication and DeviceManagementConfiguration.ReadWrite.All. EXPECTED RESULTS The pilot EDR policy assignment removed, the EICAR file deleted, and printed steps to offboard the pilot devices and resolve the test alerts. .NOTES White-label. Reversible teardown. Offboarding devices is asynchronous — allow time to reflect. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$PilotGroup = 'MDE PoC - Pilot Ring - Pilot', [string]$EdrPolicyName = 'MDE PoC EDR Onboarding - Pilot Ring - Pilot', [switch]$DeletePolicy, [string]$WorkDir = "$env:TEMP\MDE-PoC")

POCMDE-TestSignal.ps1

Purpose: Produces harmless, industry-standard test signal on an ONBOARDED PILOT DEVICE so the MDE detection pipeline has something to catch. It does two safe things: 1. Writes the EICAR anti-malware test file (a documented 68-byte harmless string that every AV recognises — it is NOT malware) to trigger Microsoft Defender Antivirus. 2. Prints the exact portal steps to launch a Microsoft BUILT-IN evaluation simulation (security.microsoft.com -> Endpoints -> Evaluation & tutorials). NO real malware, exploit, or attack tooling is downloaded or executed. -WhatIf shows what it would do without writing anything. HOW TO RUN Preview: powershell -ExecutionPolicy Bypass -File .\POCMDE-TestSignal.ps1 -WhatIf Generate

benign signal (run ON a pilot device): powershell -ExecutionPolicy Bypass -File .\POCMDE-TestSignal.ps1 EXPECTED RESULTS Microsoft Defender Antivirus detects/quarantines the EICAR file; you then run a built-in simulation from the portal. Both surface as detections for the PoC to validate. .NOTES White-label. BENIGN test signal only (EICAR + Microsoft built-in simulations). Never uses real threats. Verify the current built-in simulation catalogue in your tenant. > [CmdletBinding(SupportsShouldProcess = $true)] param([string]$Tag = 'MDE-PoC-TESTSIGNAL', [string]$WorkDir = "$env:TEMP\MDE-PoC")

How to run: Preview: powershell -ExecutionPolicy Bypass -File .\POCMDE-TestSignal.ps1 -WhatIf Generate benign signal (run ON a pilot device): powershell -ExecutionPolicy Bypass -File .\POCMDE-TestSignal.ps1 EXPECTED RESULTS Microsoft Defender Antivirus detects/quarantines the EICAR file; you then run a built-in simulation from the portal. Both surface as detections for the PoC to validate. .NOTES White-label. BENIGN test signal only (EICAR + Microsoft built-in simulations). Never uses real threats. Verify the current built-in simulation catalogue in your tenant. > [CmdletBinding(SupportsShouldProcess = $true)] param([string]$Tag = 'MDE-PoC-TESTSIGNAL', [string]$WorkDir = "$env:TEMP\MDE-PoC")

POCMDE-Validate.ps1

Purpose: Read-only validation and live-demo helper for the MDE proof of concept. Using the Microsoft 365 Defender advanced-hunting API (via Microsoft Graph Security), it confirms that the benign test signal surfaced as: - alerts (AlertInfo) — a detection reached the portal - evidence (AlertEvidence) — file/process evidence is attached - device events (DeviceEvents) — the activity is on the device timeline/hunting It also reminds you where to view the automated investigation (kept approval-gated). Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\POCMDE-Validate.ps1 -DeviceName <pilotDevice> Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A pass/fail line per capability (alert present, evidence present, device events present), plus portal paths for the timeline and automated-investigation demo. .NOTES White-label. Read-only. Advanced-hunting queries only read telemetry; they change nothing. > [CmdletBinding()] param([string]$DeviceName, [int]$LookbackHours = 6)

How to run: powershell -ExecutionPolicy Bypass -File .\POCMDE-Validate.ps1 -DeviceName <pilotDevice> Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A pass/fail line per capability (alert present, evidence present, device events present), plus portal paths for the timeline and automated-investigation demo. .NOTES White-label. Read-only. Advanced-hunting queries only read telemetry; they change nothing. > [CmdletBinding()] param([string]$DeviceName, [int]$LookbackHours = 6)